```
import math
import numpy as np
```

```
# --- Global Parameters ---
p_warp_size = 32

# --- Global Functions ---
def f_print_title(title_name):
    print(f"*---------------------------*")
    print(f"{title_name}:")
    print(f"*---------------------------*")

# Returns occupancy level of each SM scenario as a percentage (%).
# Per slide 5 - GPU HW Arch 201: memory hierarchy. Parallel performance theory. GEMM kernels.
def f_occupancy_level (active_warps_per_sm, max_warps_per_sm):
    occupancy_level = (active_warps_per_sm / max_warps_per_sm)
    return (occupancy_level*100)

# integer return of the total number of threads per SM based on threads per block and blocks per SM
def f_threads_per_sm(threads_per_block, blocks_per_sm):
    total_threads_per_sm = (threads_per_block * blocks_per_sm)
    return total_threads_per_sm

# boolean check that is TRUE when total threads per SM is below maximum threads per SM  [0,{max_threads_per_sm}]
def f_valid_threads_per_sm(total_threads_per_sm, max_threads_per_sm):
    valid_threads_per_sm = (total_threads_per_sm <= max_threads_per_sm)
    return valid_threads_per_sm

# integer return of the total registers per SM based on total threads per SM and registers per thread
def f_registers_per_sm(total_threads_per_sm, registers_per_thread):
    total_registers_per_sm = (total_threads_per_sm * registers_per_thread)
    return total_registers_per_sm

# boolean check that is TRUE when total registers per SM is below maximum registers per SM  [0,{max_registers_per_sm}]
def f_valid_registers_per_sm(total_registers_per_sm, max_registers_per_sm):
    valid_registers_per_sm = (total_registers_per_sm <= max_registers_per_sm)
    return valid_registers_per_sm
```

I. If the SM of a CUDA device can take up to 1536 threads and up to 4 thread blocks. Which of the following block configurations would result in the largest number of threads in the SM?

A. 128 threads per block
B. 256 threads per block
C. 512 threads per block
D. 1024 threads per block

```
# Following block of code will go through each X-thread block iteration and then multiply it
# by the number of threads. If the thread count goes beyond the physical limit of the
# CUDA SM (1536 threads) then discard it. Whatever value is closest to 1536 threads wins.

# # Problem Parameters
threads = [128, 256, 512, 1024] # threads per block
blocks = [1, 2, 3 , 4] # maximum blocks per SM
solution_space = np.zeros((len(blocks), len(threads)),dtype=int)

max_threads_per_sm = 1536 # maximum threads per SM
optimal_threads_per_sm = 0 # winner winner chicken dinner solution
optimal_blocks = None
optimal_threads = None

# Problem Scoreboard
def f_print_matrix_sm(solution, blocks, threads):
    print(f"Threads per SM: {solution}")
    print(f"Blocks: {blocks}")
    print(f"Threads per Block: {threads}")

# --- iterate through block*thread perturbations to explore SM solution space ---
# iterate through different block counts per SM
for i in range (len(blocks)):
    # iterate through different thread counts per block
    for j in range (len(threads)):
        solution_space[i,j] = blocks[i] * threads[j]

        # --- Find the best fit solution ---
        if (solution_space[i,j] <= max_threads_per_sm):
            if (optimal_threads_per_sm < solution_space[i,j]):
                optimal_threads_per_sm = solution_space[i,j]
                optimal_blocks = blocks[i]
                optimal_threads = threads[j]

print(f"--- Best Solution ---")
f_print_matrix_sm(optimal_threads_per_sm, optimal_blocks, optimal_threads)
```

```
--- Best Solution ---
Threads per SM: 1536
Blocks: 3
Threads per Block: 512
```

II. For a vector addition, assume that the vector length is 2000, each thread calculates one output element, and the thread block size is 512 threads. How many threads will be in the grid?

A. 2000
B. 2024
C. 2048
D. 2096

```
threads = 512 # threads per block
vector_length = 2000

blocks = math.ceil(vector_length/threads)
threads_per_grid = threads * blocks
print(f"Threads per Grid: {threads_per_grid}")
```

```
Threads per Grid: 2048
```

III. With reference to the previous question, how many warps do you expect to have divergence due to the boundary check on vector length?

A. 1
B. 2
C. 3
D. 6

```
warp_size = 32
leftovers = threads_per_grid - vector_length
diverged_warps = math.floor(leftovers / warp_size)

# Threads 2000 % 32 = 16 -> meaning there are 16 threads out of 32 that will exist under the
# 2000 vector length limit. There will 16 threads above the 2000 vector length limit that fail.
# Threads 2017-2048 will all fail the vector length limit together, so the warp doesn't diverge.
print(f"Diverged Warps: {diverged_warps}")
```

```
Diverged Warps: 1
```

IV. Assume a device that allows up to 64 blocks per SM and 2048 threads per SM. Indicate which of the following assignments per SM are possible. IN the cases in which it is possible, indicate the occupancy level.

A. 8 blocks with 128 threads each
B. 16 blocks with 64 threads each
C. 32 blocks with 32 threads each
D. 64 blocks with 32 threads each
E. 32 blocks with 64 threads each

```
# --- Parameters ---
p_max_threads_per_sm = 2048 # parameter: maximum threads per SM
p_max_warps_per_sm = p_max_threads_per_sm / p_warp_size
p_max_blocks_per_sm = 64
blocks = [8, 16, 32, 64, 32]
threads = [128, 64, 32, 32, 64]

# --- Variables ---
# SM size variables
threads_per_sm = 0 # matrix of maximum number of threads per SM
valid_threads_per_sm = 0 # matrix of valid threads per SM

# Occupancy level variables
active_warps_per_sm = 0
occupancy_level_per_sm = 0 # matrix of occupancy levels per SM

# --- Functions ---
#  Determine valid SMs and print occupancy levels
for i in range(len(blocks)):
    # Each block has an associated thread count, no need to do a perturbation.
    threads_per_sm = blocks[i] * threads[i]
    # valid SM boolean
    valid_threads_per_sm = (threads_per_sm <= p_max_threads_per_sm
                            and blocks[i] <= p_max_blocks_per_sm)

    #  Determine if you have a valid SM, and if so, print associated occupancy level
    if(valid_threads_per_sm):
        active_warps_per_sm = math.ceil(threads_per_sm / p_warp_size)
        occupancy_level_per_sm = f_occupancy_level(active_warps_per_sm, p_max_warps_per_sm)

        f_print_title("Valid SM:")
        print(f"Threads per SM: {threads_per_sm}")
        print(f"Blocks per SM: {blocks[i]}")
        print(f"Threads per Block: {threads[i]}")
        print(f"SM Occupancy Level: {occupancy_level_per_sm}%")
```

```
*---------------------------*
Valid SM::
*---------------------------*
Threads per SM: 1024
Blocks per SM: 8
Threads per Block: 128
SM Occupancy Level: 50.0%
*---------------------------*
Valid SM::
*---------------------------*
Threads per SM: 1024
Blocks per SM: 16
Threads per Block: 64
SM Occupancy Level: 50.0%
*---------------------------*
Valid SM::
*---------------------------*
Threads per SM: 1024
Blocks per SM: 32
Threads per Block: 32
SM Occupancy Level: 50.0%
*---------------------------*
Valid SM::
*---------------------------*
Threads per SM: 2048
Blocks per SM: 64
Threads per Block: 32
SM Occupancy Level: 100.0%
*---------------------------*
Valid SM::
*---------------------------*
Threads per SM: 2048
Blocks per SM: 32
Threads per Block: 64
SM Occupancy Level: 100.0%
```

E. 32 blocks with 64 threads each

V. Consider a GPU with the following hardware limits: 2048 threads per SM, 32 blocks per SM, and 64K (65,536) registers per SM. **For each of the following kernel characteristics**, specify whether the kernel can achieve full occupancy. If not, specify the limiting factor, and the maximum occupancy % achieved.

A. kernel uses 128 threads per block, and 30 registers per thread.
B. kernel uses 32 threads per block, and 29 registers per thread.
C. kernel uses 256 threads per block, and 34 registers per thread.

```
# --- Parameters ---
# problem parameters
p_max_threads_per_sm = 2048 # parameter: maximum threads per SM
p_max_warps_per_sm = (p_max_threads_per_sm / p_warp_size)
p_max_blocks_per_sm = 32
p_threads_per_warp = 32
p_max_registers_per_sm = (2**16)
# problem scenarios
threads = [128, 32, 256]
registers_per_thread = [30, 29, 34]

# --- Variables ---
# SM size variables
thread_blocks_per_sm = 0
register_blocks_per_sm = 0

# Occupancy level variables
active_warps_per_sm = 0
occupancy_level_per_sm = 0 # matrix of occupancy levels per SM

# --- Functions ---
# 1. Find the max number of blocks that can fit in each SM
for i in range(len(threads)):
    # Determine the maximum supported blocks in the SM per scenario
    registers_per_block = threads[i] * registers_per_thread[i]

    thread_blocks_per_sm = p_max_threads_per_sm / threads[i]
    register_blocks_per_sm = math.floor(p_max_registers_per_sm / registers_per_block)
    # Determine which factor is the limiting input
    active_blocks = min(thread_blocks_per_sm, register_blocks_per_sm, p_max_blocks_per_sm)

    # Occupancy math
    active_warps_per_sm = (active_blocks * threads[i]) / p_threads_per_warp
    occupancy_level_per_sm = (active_warps_per_sm / p_max_warps_per_sm) * 100

    # terminal information
    f_print_title(f"Scenario {i}")
    print(f"Thread Blocks per SM: {thread_blocks_per_sm}")
    print(f"Register Blocks per SM: {register_blocks_per_sm}")
    print(f"Max Blocks per SM: {p_max_blocks_per_sm}")
    print(f"Active Blocks (upper bound defined by lowest block count): {active_blocks}")
    print(f"Active Warps: {active_warps_per_sm}")
```

```
    print(f"Occupancy Level: {occupancy_level_per_sm}%")
```

```
*---------------------------*
Scenario 0:
*---------------------------*
Thread Blocks per SM: 16.0
Register Blocks per SM: 17
Max Blocks per SM: 32
Active Blocks (upper bound defined by lowest block count): 16.0
Active Warps: 64.0
Occupancy Level: 100.0%
*---------------------------*
Scenario 1:
*---------------------------*
Thread Blocks per SM: 64.0
Register Blocks per SM: 70
Max Blocks per SM: 32
Active Blocks (upper bound defined by lowest block count): 32
Active Warps: 32.0
Occupancy Level: 50.0%
*---------------------------*
Scenario 2:
*---------------------------*
Thread Blocks per SM: 8.0
Register Blocks per SM: 7
Max Blocks per SM: 32
Active Blocks (upper bound defined by lowest block count): 7
Active Warps: 56.0
Occupancy Level: 87.5%
```

For scenario 0 is able to achieve 100% occupancy.

For scenario 1 only 50% occupancy can be achieved, limited by the SM block count parameter of 32.

For scenario 2 only 87.5% occupancy can be achieved, limited by the number of available registers in the SM.

Analysis

Control Divergence

Calculate the impact of control divergence for the N tiles in the tiled MMULT algorithm

- In the same way as demonstrated in Lecture 4 for the M tiles
    a. refer to lecture 4 slides for details
    b. Also see PMPP section 4.5
- Discuss the behavior of the Type 1 and Type 2 blocks
    a. How many warp phases have divergence for each?
    b. What is the expected performance impact %?
        - for each Type (1/2)?
        - and the combined Total?
- Q: When will the warp divergence impact become minimal?
- Q: How does varying tile size affect arithmetic intensity?

Assumptions

Referring to lecture 4, the MMULT Kernel assumptions are:

1. 16x16 tiles and thread blocks
2. Each thread block has 8 warps ((16 threads x 16 threads) / (1 warp) = 256 threads / 32 threads = 8 warps)
3. Square 100x100 matrix
4. Each thread will go through 7 phases (ceiling of 100/16)
5. 49 thread blocks / tiles

Type 1

For an `N` matrix there are 6 full block columns (`M` does block rows) across 7 rows. Thus 6 * 7 = 42 Type 1 blocks.

At 8 warps per block, there's a total of 42 * 8 = 336 warps. So far, similar to the `M` matrix. Each warp has 7 phases (336 warps * 7 phases) = 2352 Type 1 warp-phases.

For the first six block rows, there are zero divergent warp-phases. Take threads 0-15 and 16-31 in each block for example, the threads will all process valid data. For block 7, rows 96-97 (warp 1 in block 7) will cover one warp, then rows 98-99 (warp 2 in block 7) will cover the last warp for the 100x100 MMULT matrix. Rows 100-111 will all process valid zero data. There's no divergence since all the warps in block 7 process valid data.

Type 1 Diverged Warps: 0 Type 1 Performance Impact: 0 / 2352 = 0% performance impact

Type 2

For the `N` matrix, the 7th column is where divergence occurs. There a total of (8 warps per block * 7 blocks * 7 phases) = 392 total warp-phases for column 7.

All warps in block rows 1-6 experience divergence between threads in column 0-3 versus 4-15, which causes each warp to diverge. Then for block row 7, only the two warps along rows 96-99 diverge, meanwhile warps 3-8 all process valid zero data.

For blocks 1-6, (8 warps per block x 6 phases) = 48 divergent warp-phases, but I need to account for the two divergences in block row 7 for each phase. Thus 48+2 = 50 divergent warp-phases.

Multiply the result across the 7 rows, (50 divergent warp phases * 7 block rows) = 350 total divergent warps-phases in `N`.

Type 1 Diverged Warps: 350 Type 1 Performance Impact: 350 / 392 = 89.29% performance impact

Combined Total = divergent warp-phases / total warp-phases = (350 + 0) / (2352 + 392) = 350 / 2744 = 12.76%

Q: When will the warp divergence impact become minimal?

When the `M` or `N` matrix widths are base-2 number, then there should be no warp divergence for the MMUlT algorithm.

Q: How does varying tile size affect arithmetic intensity

Based on this assignment, a larger tile means that more phases can be computed for the same thread-block. Thus I would say increasing tile size increases arithmetic intensity since more operations are performed with only one memory traffic byte for example, versus a smaller tile size would mean fewer phases and thus fewer operations per memory traffic byte.

Control Divergence

The launch order is single divergent, single uniform, multiple divergent, multiple uniform.

Occupancy results

Case	Theoretical occupancy (%)	Achieved occupancy (%)	Theoretical active warps per SM	Achieved active warps per SM
Single: divergent	50.00	18.66	16	5.97
Single: uniform	50.00	18.65	16	5.97
Multiple: divergent	50.00	18.01	16	5.76
Multiple: uniform	50.00	17.93	16	5.74

Warp Results

Case	Duration (µs)	Divergent-branch counter	Avg. active threads per warp	Avg. threads not predicated off per warp
Single: divergent	2.528	256	21.69	21.25
Single: uniform	3.232	0	32.00	32.00
Multiple: divergent	1.824	256	29.05	26.95
Multiple: uniform	1.824	0	32.00	32.00

Other GPU Speed of Light metrics

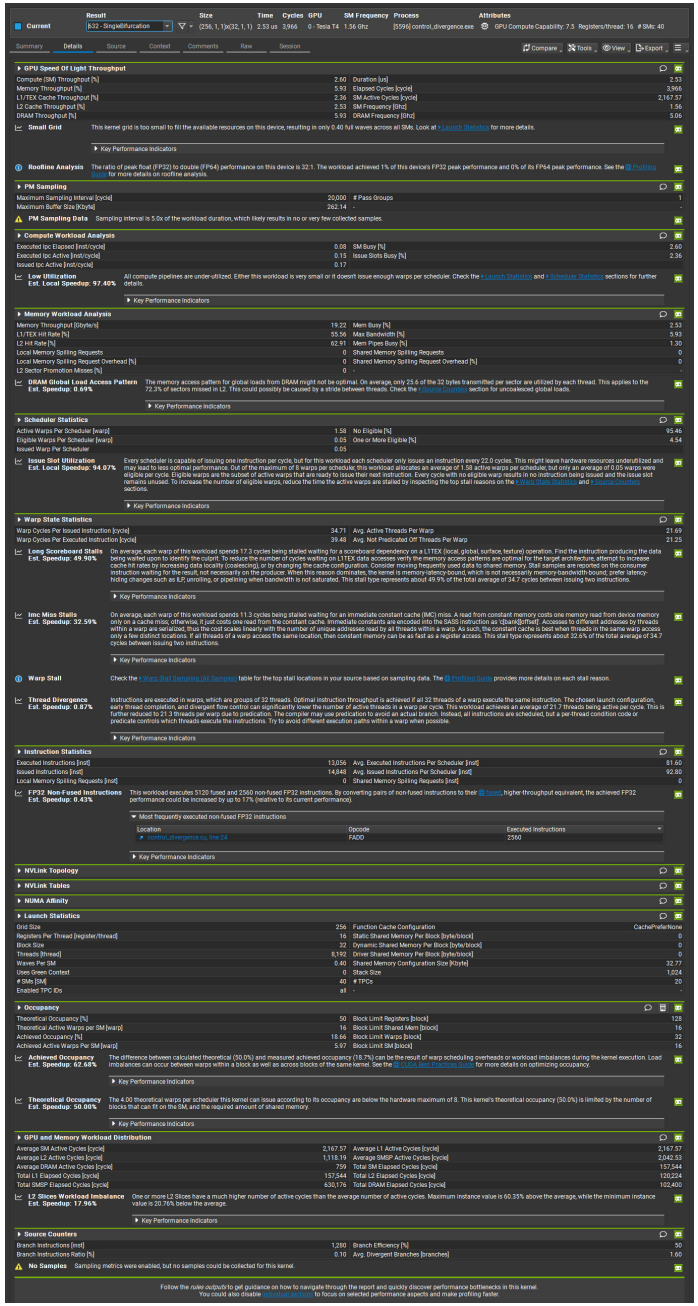
Case	Compute (SM) throughput (%)	Memory throughput (%)	DRAM throughput (%)
Single: divergent	2.60	5.93	5.93
Single: uniform	1.61	10.18	10.18
Multiple: divergent	1.58	2.21	0.55
Multiple: uniform	1.36	2.23	0.57

How does Nsight Compute know when Divergence is happening?

Positive values in the divergent-branch counter and reduced active thread averages help the tool determine divergent cases. Single divergent averages

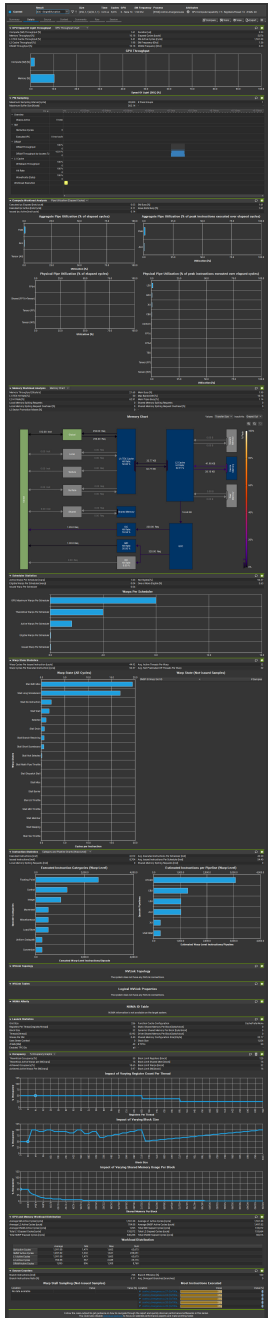
were 21.69 active and 21.25 active threads, meanwhile multiple divergent averages were 29.05 and 26.95. The uniform averages were 32.00.

Single bifurcation: divergent

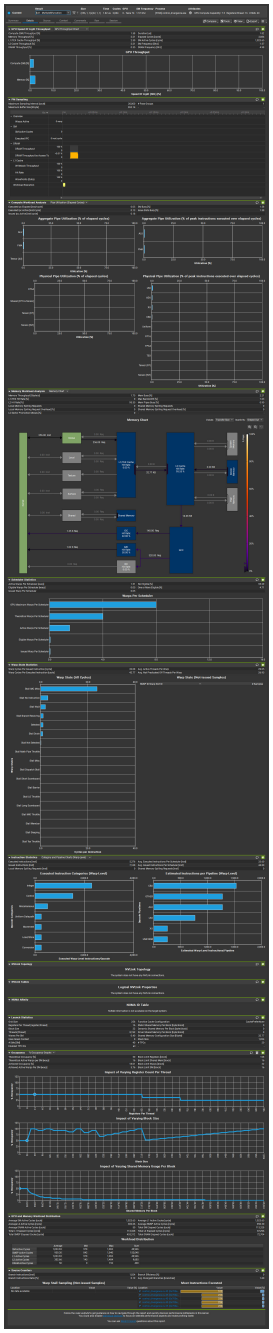


You can ask Nsight Copilot questions about this report.

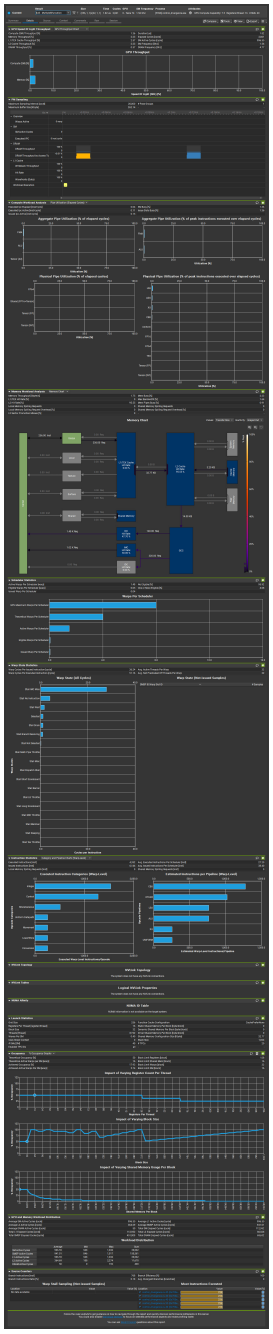
Single bifurcation: uniform



Multiple bifurcation: divergent



Multiple bifurcation: uniform



MMULT Variants

Printed output from the MMULT release build.

MMULT Tile 8

```
D:\PhotonUser\Downloads\uw-main\uw-main\eep524_applied_high_performance_gpu_computing\assignments\hw3>mmu
lt.exe

Output: 100 x 100, inner size: 100, tile width: 8
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 1000 x 1000, inner size: 1000, tile width: 8
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 2000 x 3000, inner size: 2500, tile width: 8
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
```

```
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
```

MMULT Tile12

```
D:\PhotonUser\Downloads\uw-main\uw-main\eep524_applied_high_performance_gpu_computing\assignments\hw3>mmu
lt.exe

Output: 100 x 100, inner size: 100, tile width: 12
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 1000 x 1000, inner size: 1000, tile width: 12
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 2000 x 3000, inner size: 2500, tile width: 12
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
```

```
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
```

MMULT Tile16

```
:\PhotonUser\Downloads\uw-main\uw-main\eep524_applied_high_performance_gpu_computing\assignments\hw3>mmul
t.exe

Output: 100 x 100, inner size: 100, tile width: 16
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 1000 x 1000, inner size: 1000, tile width: 16
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 2000 x 3000, inner size: 2500, tile width: 16
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
```

```
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
```

MMULT Tile32

```
D:\PhotonUser\Downloads\uw-main\uw-main\eep524_applied_high_performance_gpu_computing\assignments\hw3>mmu
lt.exe

Output: 100 x 100, inner size: 100, tile width: 32
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 10000
  Maximum error fraction (finite, nonzero CPU values): 2.27303e-07
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 1000 x 1000, inner size: 1000, tile width: 32
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 1000000
  Maximum error fraction (finite, nonzero CPU values): 8.91377e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0

Output: 2000 x 3000, inner size: 2500, tile width: 32
Tolerances: relative = 0.0001, zero reference = 1e-05
Computing CPU reference...
Naive GPU: PASS
```

```
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
Tiled GPU: PASS
  Mismatches: 0 out of 6000000
  Maximum error fraction (finite, nonzero CPU values): 7.08061e-08
  Zero CPU values checked with zero tolerance: 0
  Non-finite output pairs: 0
```

Results

Maximum single-element abs error fraction: 100x100 case = 2.27303e-07

Output matrix: 100 × 100, K = 100

Kernel	Tile width / block shape	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
Naive	8 / 8×8	10.3203	228.102	23.89	0.008768
Naive	12 / 12×12	10.9977	265.957	27.57	0.007520
Naive	16 / 16×16	6.7509	233.209	27.56	0.008576
Naive	32 / 32×32	13.0835	180.115	88.55	0.011104
Tiled	8 / 8×8	9.8829	183.616	26.25	0.011328
Tiled	12 / 12×12	13.5597	221.311	33.18	0.009760
Tiled	16 / 16×16	16.5017	246.479	30.42	0.009088
Tiled	32 / 32×32	21.4765	217.391	99.37	0.011776

Output matrix: 1000 × 1000, K = 1000

Kernel	Tile width / block shape	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
Naive	8 / 8×8	29.3729	582.088	96.89	3.435904
Naive	12 / 12×12	26.6318	565.324	88.02	3.537792
Naive	16 / 16×16	21.6077	720.951	98.59	2.774112
Naive	32 / 32×32	17.2530	779.195	98.38	2.566752
Tiled	8 / 8×8	28.6399	673.753	98.89	2.968448
Tiled	12 / 12×12	23.8076	682.136	92.99	2.955424
Tiled	16 / 16×16	24.7931	848.039	99.19	2.377248
Tiled	32 / 32×32	19.5223	861.141	100.18	2.378240

Output matrix: 2000 × 3000, K = 2500

Kernel	Tile width / block shape	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
Naive	8 / 8×8	1.1259	284.417	90.07	105.479040
Naive	12 / 12×12	1.9575	324.305	90.40	92.505600
Naive	16 / 16×16	4.2181	549.977	99.18	54.547776
Naive	32 / 32×32	9.7721	719.499	98.74	41.695680

Kernel	Tile width / block shape	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
Tiled	8 / 8×8	2.2105	501.190	99.75	59.953280
Tiled	12 / 12×12	3.2384	552.644	93.56	54.458240
Tiled	16 / 16×16	4.2548	737.940	100.02	40.848832
Tiled	32 / 32×32	11.4933	839.133	99.90	36.151584

FMA comparison: output 2000 × 3000, K = 2500

Build	Tile width	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
Normal (default FMA enabled)	32	11.4933	839.133	99.90	36.151584
FMA disabled (`--fmad=false`)	32	11.4795	810.707	100.06	37.419200

For the largest tiled case, disabling FMA increased measured duration by ~1.3 ms and decreased achieved throughput by 3.39%.

Measured AI changed by -0.12%, so it was essentially unchanged, and achieved occupancy stayed approximately ~100%.

All matched 32×32 comparisons

Normal rows come from `mmult_tile32.ncu-repz` ; disabled rows come from `mmult_tile32_no_fmp.ncu-repz` .

Output matrix	Kernel	FMA Disabled	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
100 × 100	Naive	No	13.0835	180.115	88.55	0.011104
100 × 100	Naive	Yes	12.9695	180.636	87.11	0.011072
100 × 100	Tiled	No	21.4765	217.391	99.37	0.011776
100 × 100	Tiled	Yes	21.3732	212.766	100.33	0.012032
1000 × 1000	Naive	No	17.2530	779.195	98.38	2.566752
1000 × 1000	Naive	Yes	17.7090	783.444	97.65	2.552832
1000 × 1000	Tiled	No	19.5223	861.141	100.18	2.378240
1000 × 1000	Tiled	Yes	19.9583	835.760	99.61	2.450464
2000 × 3000	Naive	No	9.7721	719.499	98.74	41.695680
2000 × 3000	Naive	Yes	9.7078	714.737	97.93	41.973472
2000 × 3000	Tiled	No	11.4933	839.133	99.90	36.151584

Output matrix	Kernel	FMA Disabled	AI (FLOP/byte)	Throughput (GFLOP/s)	Achieved occupancy (%)	Duration (ms)
2000 × 3000	Tiled	Yes	11.4795	810.707	100.06	37.419200

Which configurations provide the best performance?

The winners below compare all eight recorded combinations (naive/tiled × four block shapes) for each matrix size.

Output matrix	Highest AI	Highest throughput	Highest achieved occupancy	Shortest kernel duration
100 × 100	Tiled, 32×32: 21.4765 FLOP/byte	Naive, 12×12: 265.957 GFLOP/s	Tiled, 32×32: 99.37%	Naive, 12×12: 0.007520 ms
1000 × 1000	Naive, 8×8: 29.3729 FLOP/byte	Tiled, 32×32: 861.141 GFLOP/s	Tiled, 32×32: 100.18%	Tiled, 16×16: 2.377248 ms
2000 × 3000	Tiled, 32×32: 11.4933 FLOP/byte	Tiled, 32×32: 839.133 GFLOP/s	Tiled, 16×16: 100.02%	Tiled, 32×32: 36.151584 ms

For the tiled implementation alone, the fastest recorded configurations are:

Output matrix	Fastest tiled configuration	Duration (ms)
100 × 100	16×16	0.009088
1000 × 1000	16×16	2.377248
2000 × 3000	32×32	36.151584

Interpretation of the remaining performance questions

Does the best configuration vary with matrix size?

Yes. Overall, naive 12×12 is fastest for 100×100, tiled 16×16 for 1000×1000, and tiled 32×32 for 2000×3000. For the small matrix, the extra tiled loading and synchronization can outweigh the benefit of reuse. This is consistent with the timings, rather than proof of a single bottleneck.

Are any of the measured cases shown to be FP32 compute-bound?

None were at the FP32 bound because throughput is only about 2.3%–10.8% of the peak calculated from the report counters.

Why does the measured AI not always increase with tile width?

The larger tile could have required more threads handling boundary padding in this run, or there could have been more traffic to memory for the larger tiles on real hardware. In this case, the number of operations would go down for the bytes loaded, thus the lower AI than expected.

Does maximizing threads per block guarantee the highest occupancy?

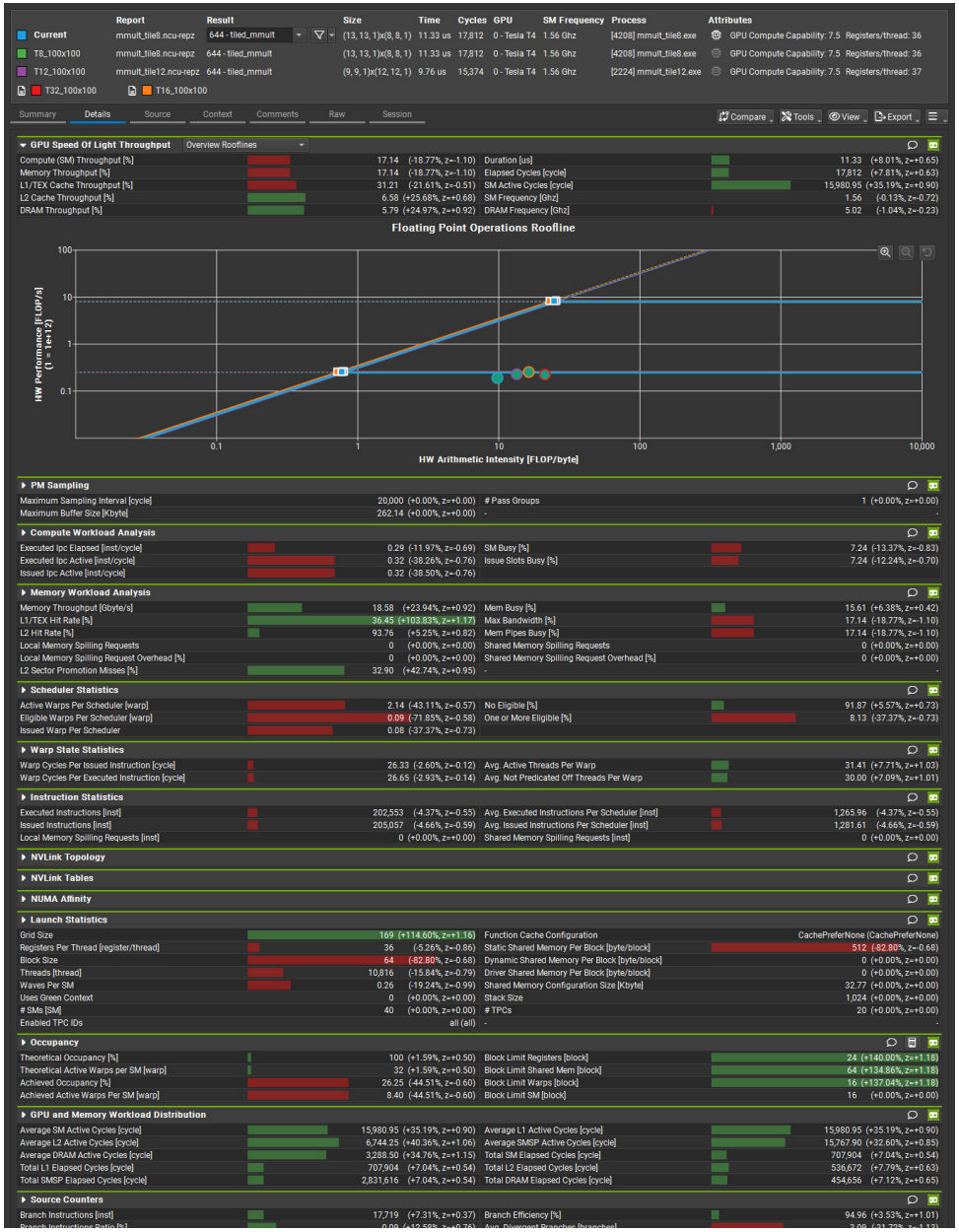
Not for all cases, for example if processing a 100x100 matrix than occupancy can be higher with different TILE_WIDTH configurations.

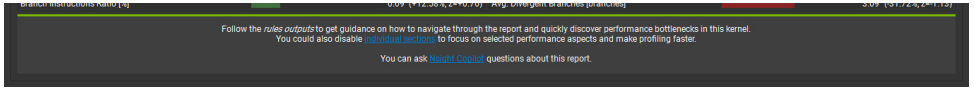
Is transposing N worth its overhead?

No, it's not worth it. It breaks the current matrix algorithm and would require the MMULT kernel to be re-written to account for the new `N` matrix indexing.

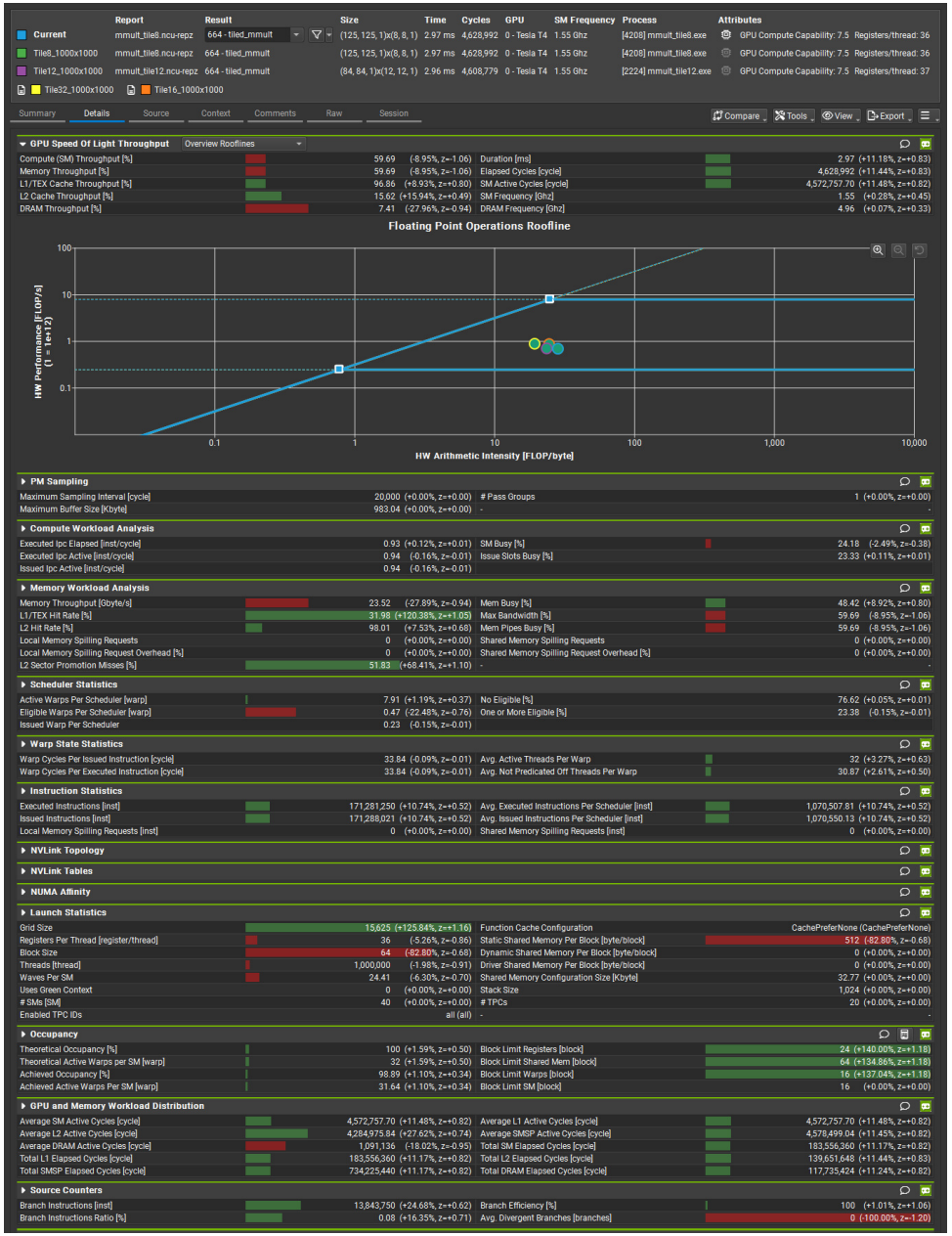
Nsight Compute MMULT Data

Output matrix: 100 × 100





Output matrix: 1000 × 1000



Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable individual sections to focus on selected performance aspects and make profiling faster.

You can ask Nsight Copilot questions about this report.

Output matrix: 2000 × 3000